

Guía rápida: subir clase9ws Spring Boot a AWS EC2

API REST + MySQL Northwind + pruebas desde navegador

Esta guía resume exactamente el flujo usado para publicar el proyecto clase9ws20232 en una instancia EC2. Está pensada para laboratorio: comandos cortos, copiables y con explicación simple.

1. Idea general

Para que la API funcione en la nube se necesitan dos cosas:

Componente	Función
app.jar	Es la aplicación Spring Boot. Expone rutas como /product.
MySQL en Docker	Guarda la base de datos northwind con tablas como products, categories y suppliers.

Flujo: navegador o consola -> EC2:8080 -> Spring Boot -> MySQL Northwind.

2. Crear o usar una instancia EC2

Si no tienes instancia, crea una en AWS EC2. Para este laboratorio usamos Amazon Linux con Java 17 y Docker.

En el Security Group se deben abrir como mínimo:

- Puerto 22: SSH para conectarte a la instancia.
- Puerto 8080: para acceder a la API Spring Boot desde el navegador.
- Puerto 80: opcional si luego usas Nginx.

3. Conectarse por SSH desde Windows

Comando base:

```
ssh -i D:\lab11_prueba.pem ec2-user@100.55.18.227
```

Si sale el error "UNPROTECTED PRIVATE KEY FILE" o "bad permissions", arregla permisos de la llave .pem en PowerShell:

```
cd D:\
icacls .\lab11_prueba.pem /inheritance:r
icacls .\lab11_prueba.pem /remove "NT AUTHORITY\Authenticated Users"
icacls .\lab11_prueba.pem /remove "BUILTIN\Users"
icacls .\lab11_prueba.pem /grant:r "$($env:USERNAME):R"
ssh -i .\lab11_prueba.pem ec2-user@100.55.18.227
```

Nota: Cambia la IP y el nombre de la llave por los tuyos si son diferentes.

4. Verificar Java en EC2

```
java -version
```

Debe salir Java 17. Si no está instalado:

```
sudo yum install java-17-amazon-corretto -y  
java -version
```

5. Instalar Docker si no existe

```
docker --version
```

Si dice command not found, instalar Docker:

```
sudo yum update -y  
sudo yum install docker -y  
sudo systemctl enable docker  
sudo systemctl start docker  
sudo usermod -aG docker ec2-user  
exit
```

Luego vuelve a entrar por SSH y prueba:

```
docker ps
```

6. Crear MySQL Northwind en Docker

Crear el contenedor MySQL con base de datos northwind:

```
docker run --name mysql-northwind \  
-e MYSQL_ROOT_PASSWORD=12345678 \  
-e MYSQL_DATABASE=northwind \  
-p 3306:3306 \  
-d mysql:8.0
```

Verificar que esté corriendo:

```
docker ps
```

Probar conexión a MySQL:

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "SHOW DATABASES;"
```

7. Subir y cargar northwind.sql

Desde tu PC, sube el archivo SQL a la instancia. Si tu archivo se llama northwindd.sql, no hay problema; puedes subirlo con ese nombre o renombrarlo al subir:

```
scp -i D:\lab11_prueba.pem .\northwindd.sql ec2-user@100.55.18.227:~/clase9ws/northwind.sql
```

En EC2, crea la carpeta si aún no existe:

```
mkdir -p ~/clase9ws
```

Cargar el SQL dentro de la base northwind:

```
docker exec -i mysql-northwind mysql -uroot -p12345678 northwind < ~/clase9ws/northwind.sql
```

Verificar tablas:

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "USE northwind; SHOW TABLES;"
```

8. Error típico: tabla products no existe

En Linux, MySQL puede distinguir mayúsculas y minúsculas. Si la app busca products pero la BD creó Products, saldrá error 500.

Error visto:

```
Table 'northwind.products' doesn't exist
```

Primero revisa las tablas:

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "USE northwind; SHOW TABLES;"
```

Si aparecen Products, Categories y Suppliers con mayúscula, renómbralas a minúscula:

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "USE northwind; RENAME TABLE Products TO products;"
```

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "USE northwind; RENAME TABLE Categories TO categories;"
```

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "USE northwind; RENAME TABLE Suppliers TO suppliers;"
```

Verifica que existan en minúscula:

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "USE northwind; SHOW TABLES;"
```

```
docker exec -it mysql-northwind mysql -uroot -p12345678 -e "USE northwind; SELECT COUNT(*) FROM products;"
```

9. Generar el .jar en IntelliJ

En la terminal de IntelliJ, dentro del proyecto clase9ws20232:

```
mvn clean package -DskipTests
```

El archivo se genera en target. Debe verse algo como:

```
target/clase9ws20232-0.0.1-SNAPSHOT.jar
```

10. Subir el .jar a EC2

Desde PowerShell, estando en la raíz del proyecto:

```
scp -i D:\lab11_prueba.pem .\target\clase9ws20232-0.0.1-SNAPSHOT.jar  
ec2-user@100.55.18.227:~/clase9ws/app.jar
```

En EC2, verifica que llegó:

```
cd ~/clase9ws  
ls -lh
```

11. Configuración esperada de Spring Boot

El application.properties del proyecto debe apuntar al MySQL de la misma instancia. Como Spring Boot corre en EC2 y MySQL también está publicado en el puerto 3306, se puede usar 127.0.0.1:

```
spring.datasource.url=jdbc:mysql://127.0.0.1:3306/northwind  
spring.datasource.username=root  
spring.datasource.password=12345678  
server.port=8080
```

Nota: Si cambiaste la contraseña de MySQL, cambia también `spring.datasource.password` y vuelve a generar/subir el `.jar`.

12. Ejecutar la API en EC2

```
cd ~/clase9ws  
java -jar app.jar
```

Si todo está bien, debe salir algo parecido a:

```
HikariPool-1 - Start completed.  
Tomcat started on port(s): 8080  
Started Clase9ws20232Application
```

Eso significa que Spring Boot levantó y conectó a MySQL.

13. Probar la API desde navegador

Abrir en navegador:

```
http://100.55.18.227:8080/product
```

También puedes probar desde la misma EC2:

```
curl http://localhost:8080/product
```

14. Comandos para probar desde Console del navegador

Abrir F12 -> Console y pegar:

Listar productos

```
fetch("http://100.55.18.227:8080/product")  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error("Error:", error));
```

Listar productos en tabla

```
fetch("http://100.55.18.227:8080/product")  
  .then(response => response.json())  
  .then(data => console.table(data))
```

```
.catch(error => console.error("Error:", error));
```

Obtener producto por ID

```
fetch("http://100.55.18.227:8080/product/1")  
  
  .then(response => response.json())  
  
  .then(data => console.log(data))  
  
  .catch(error => console.error("Error:", error));
```

Crear producto

```
fetch("http://100.55.18.227:8080/product?fetchId=true", {  
  
  method: "POST",  
  
  headers: {  
  
    "Content-Type": "application/json"  
  
  },  
  
  body: JSON.stringify({  
  
    productName: "Producto desde nube",  
  
    supplier: { id: 1 },  
  
    category: { id: 1 },  
  
    quantityPerUnit: "10 unidades",  
  
    unitPrice: 25,  
  
    unitsInStock: 20,  
  
    unitsOnOrder: 0,  
  
    reorderLevel: 5,  
  
    discontinued: false  
  
  })  
  
})  
  
  .then(response => response.json())  
  
  .then(data => console.log(data))  
  
  .catch(error => console.error("Error:", error));
```

15. Dejar la API corriendo aunque cierres la terminal

Para laboratorio rápido, puedes usar nohup:

```
cd ~/clase9ws  
nohup java -jar app.jar > app.log 2>&1 &
```

Ver logs:

```
tail -f ~/clase9ws/app.log
```

Ver proceso Java:

```
ps aux | grep app.jar
```

Detenerlo si hace falta:

```
pkill -f app.jar
```

16. Errores comunes y solución rápida

Error	Qué significa	Solución
Bad permissions .pem	La llave SSH está demasiado abierta en Windows.	Arreglar permisos con icacfs.
Connection refused MySQL	Spring Boot no encuentra MySQL en 3306.	Crear/iniciar mysql-northwind en Docker.
Whitelabel 500	La app levantó, pero falló al consultar.	Mirar log de Java y validar tablas.
Table 'northwind.products' doesn't exist	La tabla no existe o está con mayúscula.	SHOW TABLES y renombrar Products -> products.
No carga desde navegador	Puerto 8080 cerrado o app apagada.	Abrir Security Group y revisar java -jar/app.log.

17. Resumen para explicar en el laboratorio

Primero se creó una instancia EC2 y se accedió por SSH. Luego se instaló Java 17 para ejecutar Spring Boot. Después se levantó MySQL en Docker con la base northwind y se cargó el archivo SQL. En local se generó el .jar con Maven, se subió a EC2 con scp y se ejecutó con java -jar app.jar. Finalmente, se probó la API en el navegador usando la ruta /product y también desde Console con fetch.